

学生任务手册 (Part B)

B1. 任务

给你一个由强模型 (Sonnet) 生成的 (详细提示词 → SVG 徽标) 配对数据集。你需要完成两件事：

1. 设计一个验证 + 奖励 (reward) 函数。从基线 `student_kit/reward.py` 出发，用程序化的方式定义如何判断一个生成的徽标是否合格——有效性、是否正确闭合、配色/元素数量是否合理、坐标是否越界、是否覆盖提示词关键词、是否退化输出等等，只要你能说明理由即可。这是你的 训练代理指标 (training proxy)，其本身也是一个评分项。
2. 用 LoRA 微调小模型 Gemma 3 270M，使其在给定详细提示词后能画出一个有效的 SVG 徽标——以你的 reward 为优化目标。然后衡量：微调是否比它自己的基座模型有提升？

我们并不要求你达到 Sonnet、甚至 4B 模型的水平。评分依据是相对基座 270M 的提升、你的 reward 设计质量，以及你的分析质量。

徽标看起来很差是正常的——不用担心。Gemma 3 270M 是一个极小的模型（比绘制目标的那个模型小几百倍）。即便你训练得很好，它生成的徽标也会简单、粗糙，而且常常与提示词并不吻合。这是预期之内的，不会降低你的分数。你的分数来自相对基座模型的提升、你的 reward 设计和分析——而不是输出好不好看。一个能稳定产出「有效但朴素」徽标、并被很好地度量与解释的模型，就是很强的结果。

重要：老师会用一个冻结的、留出的评测指标（程序化分数 + 在私有测试集上的 Sonnet 视觉评审）来给你的适配器打分，这个指标你看不到、也不会用于训练。你自己的 reward 只是你所优化的代理指标；老师的才是真实标准。作业的一部分，就是观察两者是否一致——还是你「钻了代理指标的空子」（Goodhart 效应：代理分数上去了，真实质量却没跟上）。

B2. 你会拿到什么

- `train.jsonl`、`valid.jsonl`——提示词 → SVG 配对 (chat 格式)，从 GitHub 数据仓库 [roboticcam/logo-detailed-prompt](https://github.com/roboticcam/logo-detailed-prompt) 克隆 (直接 `git clone`；无需数据集平台、无需 Anthropic key)。
- `student_kit/`——一份训练脚本、一份自评脚本、一份基线 `reward.py` (供你扩展)，以及本说明。
- (留出的测试集和老师冻结的评分 reward 是私有的；老师用它们给你的适配器打分。)

B3. 环境

- 算力：你不需要租用或付费购买 GPU。Gemma 3 270M 非常小——百度 AI Studio ([ais-tudio.baidu.com](https://studio.baidu.com)) 的免费 GPU 额度就绰绰有余 (一块免费的 Colab 级 GPU、甚至一台笔记本都够用)。
- 基座模型：从 ModelScope (魔搭, modelscope.cn) 下载 Gemma 3 270M——例如 `modelscope download --model <gemma-3-270m repo> --local_dir ./gemma3-270m`。不要用 Hugging Face (被墙/慢)。
- 框架：推荐使用 `ms-swift` (ModelScope 官方 LoRA 工具，一条命令即可)；`transformers` + PEFT 是备选方案。
- 数据：从 github.com/roboticcam/logo-detailed-prompt 克隆 `train.jsonl` + `valid.jsonl` (`git clone https://github.com/roboticcam/logo-detailed-prompt`)——直接克隆，无需数据集平台、无需 LFS。

B4. 训练

0. 先设计你的 reward。在基线之上扩展 `student_kit/reward.py`：决定哪些程序化检查定义了一个「好」徽标，并为每一项给出理由。这个 reward 就是你后续优化和分析的对象——请把它当作项目的头等大事，而不是事后补充。
1. 用提供的 `train_swift.yaml`（或 `train_peft.py`）在 `train.jsonl` 上做 LoRA 微调。损失只在 SVG 部分计算（`mask_prompt`）。
2. 从默认配置开始，然后 做实验：LoRA rank、学习率、SVG 长度上限、训练样本数量，以及 早停（关注验证集 loss——小数据很快就会过拟合）。

B5. 自评

运行 `python student_kit/eval_self.py`，用 基座 和你 微调后的模型 分别为 `valid.jsonl` 生成徽标（使用固定的解码设置）。它会用 你的 `reward.py` 给每个徽标打分并写出 `results.json`。你的核心数字就是验证集上的 基座 vs 微调——由 你所设计的 reward 来衡量。

B6. 提交什么、提交到哪里

推送一个 GitHub 仓库（github.com），其中包含：

<code>adapter/</code>	<code>adapter_config.json + adapter_model.safetensors</code>	（几 MB——普通 git，无需 LF）
<code>reward.py</code>	你的验证/奖励函数（在基线之上扩展）	
<code>train_config.yaml</code>	你确切的超参数	
<code>results.json</code>	你自评的验证集分数（基座 vs 微调，用你的 reward）	
<code>report.md</code>	你的分析（主要的评分产物）	

也可以（可选）把适配器推送到 ModelScope 模型仓库，体验真实的 ML Hub——但评分渠道是 GitHub 仓库。

在 `report.md` 中说明：你尝试了什么、你的「基座 vs 微调」数字、为什么 有或没有提升，以及几个示例徽标（前后对比）。诚实的负面结果，只要推理到位，也能拿到好分数。

B7. 评分标准

- 相对基座的提升——有效性 Δ 和保真度 Δ （老师会在私有测试集上重跑一个冻结评测，含 Sonnet 视觉评审）。
- 可复现性——你提交的适配器能加载，并大致复现你的 `results.json`。
- 报告质量——对结果 为什么会是这样的洞察。
- 代码清晰度。

不要求 打败 Sonnet。一个被充分理解的「小幅提升 / 无提升」结果也能及格。